

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 8

SCHEMA NORMALIZATION

1. Aim

To normalize a relational schema step by step from 1NF to 2NF, 3NF, BCNF and 4NF using functional and multivalued dependencies, and to verify in MySQL that the anomalies are eliminated and the decomposition is lossless.

2. Problem Statement

The examination section of a college maintains results in a single register in which each student row lists the student's department and head of department, followed by a repeating group of courses with course title, instructor and marks (Section 8). The register suffers from redundancy and anomalies. Normalize it up to BCNF, and use two further relations - TEACHING(student, subject, instructor) and COURSE_BOOK(course_id, instructor, textbook) - to illustrate BCNF and 4NF. Implement every stage in MySQL and verify the elimination of anomalies.

Note: The course document lists only the title of this practical. The relations and data used here were prepared for this report to illustrate normalization; they are not official university questions. All names are fictitious.

3. Objectives

1. To identify functional dependencies, candidate keys and prime attributes of a relation.
2. To recognise update, insertion and deletion anomalies.
3. To decompose a relation into 1NF, 2NF, 3NF and BCNF.
4. To understand multivalued dependencies and Fourth Normal Form.
5. To verify lossless decomposition and elimination of anomalies using SQL.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files

5. Theory

5.1 Normalization

Normalization is the process of decomposing a relation that contains redundant data into smaller, well-structured relations, using functional and multivalued dependencies, so that each fact is stored only once. The decomposition must be **lossless** (the original relation can be rebuilt by a natural join) and should preferably be **dependency preserving**.

5.2 Anomalies Caused by Redundancy

- **Update anomaly:** the same fact is stored in many rows, so changing it in only some rows makes the data inconsistent.
- **Insertion anomaly:** a fact cannot be stored unless some unrelated fact is also known (e.g. a new course cannot be recorded until a student enrolls in it).
- **Deletion anomaly:** deleting one fact unintentionally deletes another (deleting the only enrollment of a course loses the course and its instructor).

5.3 Functional Dependency (FD)

In a relation R , $X \rightarrow Y$ (X functionally determines Y) holds if any two tuples that agree on X also agree on Y . A **candidate key** is a minimal set of attributes that determines all attributes; attributes belonging to some candidate key are **prime**. An FD $X \rightarrow Y$ is **partial** if Y depends on a proper subset of a candidate key, and **transitive** if $X \rightarrow Z$ and $Z \rightarrow Y$ where Z is not a key. Armstrong's axioms (reflexivity, augmentation, transitivity) are used to derive further FDs.

5.4 Normal Forms

Normal form	Condition
1NF	All attribute values are atomic; no repeating groups or multivalued cells.
2NF	1NF and no non-prime attribute is partially dependent on any candidate key.
3NF	2NF and no non-prime attribute is transitively dependent on a candidate key; i.e. for every non-trivial $X \rightarrow A$, X is a superkey or A is prime.
BCNF	For every non-trivial FD $X \rightarrow Y$, X is a superkey (stricter than 3NF).
4NF	BCNF and for every non-trivial multivalued dependency $X \twoheadrightarrow Y$, X is a superkey.

5.5 Multivalued Dependency (MVD)

$X \twoheadrightarrow Y$ (X multi-determines Y) holds when, for each value of X , the set of Y values is independent of the values of the remaining attributes Z . MVDs arise when one relation stores two independent multivalued facts about the same entity; every combination of the two sets must then be stored, which creates redundancy even when the relation is in BCNF. By the decomposition theorem, $R(X, Y, Z)$ with $X \twoheadrightarrow Y$ is decomposed losslessly into $R_1(X, Y)$ and $R_2(X, Z)$.

5.6 Verifying a Decomposition in SQL

An FD $X \rightarrow Y$ can be tested on stored data with `GROUP BY X HAVING COUNT(DISTINCT Y) > 1` (any row returned is a violation). A decomposition is checked to be lossless by joining the new relations and comparing the result with the original relation using `EXCEPT` in both directions. MySQL supports `EXCEPT` and `INTERSECT` from version 8.0.31; in older versions `NOT EXISTS` is used.

6. Procedure / Algorithm

The steps below were carried out on the sample relation of Section 2; Section 7 implements and verifies each step in MySQL.

6.1 Step 1 - Unnormalized form (UNF) to 1NF

In the UNF relation each student row contains a repeating group of courses (Section 8). The repeating group is flattened so that every cell holds one value, giving:

RESULT_1NF(roll_no, student_name, dept_code, dept_name, hod_name, course_id, course_title, instr_id, instr_name, marks)

Candidate key: {roll_no, course_id}. Functional dependencies identified:

FD	Dependency	Type
FD1	roll_no, course_id $\rightarrow$ marks	Full dependency on the key
FD2	roll_no $\rightarrow$ student_name, dept_code	Partial (on part of the key)
FD3	course_id $\rightarrow$ course_title, instr_id	Partial (on part of the key)
FD4	dept_code $\rightarrow$ dept_name, hod_name	Transitive (roll_no $\rightarrow$ dept_code $\rightarrow$...)
FD5	instr_id $\rightarrow$ instr_name	Transitive (course_id $\rightarrow$ instr_id $\rightarrow$...)

6.2 Step 2 - 1NF to 2NF

FD2 and FD3 are partial dependencies, so RESULT_1NF is not in 2NF. The attributes that depend on only part of the key are moved to new relations with that part as the key:

- STUDENT_2NF(roll_no, student_name, dept_code, dept_name, hod_name)
- COURSE_2NF(course_id, course_title, instr_id, instr_name)
- RESULT_2NF(roll_no, course_id, marks)

6.3 Step 3 - 2NF to 3NF

STUDENT_2NF contains the transitive dependency roll_no $\rightarrow$ dept_code $\rightarrow$ dept_name, hod_name and COURSE_2NF contains course_id $\rightarrow$ instr_id $\rightarrow$ instr_name. Removing them gives the 3NF schema:

- DEPARTMENT(dept_code, dept_name, hod_name)
- INSTRUCTOR(instr_id, instr_name)
- STUDENT(roll_no, student_name, dept_code*)
- COURSE(course_id, course_title, instr_id*)

- RESULT(roll_no*, course_id*, marks)

In each of these relations the only determinant is the primary key, so they are also in BCNF and, having no multivalued dependencies, in 4NF.

6.4 Step 4 - A relation in 3NF but not in BCNF

Consider TEACHING(student, subject, instructor) with the rules: for each subject a student has exactly one instructor, and each instructor teaches only one subject (a subject may have several instructors). FDs: {student, subject} → instructor and instructor → subject. Candidate keys are {student, subject} and {student, instructor}, so all attributes are prime and the relation is in 3NF. However, in instructor → subject the determinant is not a superkey, so it violates BCNF. Decomposition on this FD gives:

- INSTRUCTOR_SUBJECT(instructor, subject)
- STUDENT_INSTRUCTOR(student, instructor*)

The decomposition is lossless (common attribute instructor is the key of INSTRUCTOR_SUBJECT) but does not preserve {student, subject} → instructor, which must be checked by the application or a trigger - a known trade-off of BCNF.

6.5 Step 5 - A relation in BCNF but not in 4NF

COURSE_BOOK(course_id, instructor, textbook) records that a course is taught by several instructors and uses several textbooks, the two facts being independent. There is no non-trivial FD, the only key is all three attributes, so the relation is in BCNF. But course_id → → instructor and course_id → → textbook hold, and course_id is not a superkey, so it is not in 4NF: every instructor must be stored with every textbook (2 x 2 = 4 rows for CS301). Decomposition gives:

- COURSE_INSTRUCTOR(course_id, instructor)
- COURSE_TEXTBOOK(course_id, textbook)

6.6 Implementation Steps in MySQL

1. Create normalization_db and the 1NF relation; insert the sample data.
2. Verify the FDs on the data and measure the redundancy.
3. Demonstrate the update, insertion and deletion anomalies (inside transactions that are rolled back).
4. Create the 2NF and 3NF relations with keys and foreign keys and populate them with INSERT ... SELECT DISTINCT.
5. Verify the lossless join using row counts and EXCEPT.
6. Repeat the anomaly operations on the 3NF schema and observe that they no longer occur.
7. Create, test and decompose the BCNF and 4NF examples and verify their decompositions.

7. SQL Queries / Program

Q1: Create the 1NF relation RESULT_1NF and load the sample data

```

DROP DATABASE IF EXISTS normalization_db;
CREATE DATABASE normalization_db;
USE normalization_db;
CREATE TABLE result_1nf (
    roll_no      CHAR(3),
    student_name VARCHAR(30),
    dept_code     VARCHAR(5),
    dept_name     VARCHAR(30),
    hod_name      VARCHAR(30),
    course_id     CHAR(5),
    course_title  VARCHAR(30),
    instr_id      CHAR(3),
    instr_name    VARCHAR(30),
    marks         INT,
    PRIMARY KEY (roll_no, course_id)
);
INSERT INTO result_1nf VALUES
('S01', 'Aarav Sharma', 'CSE', 'Computer Engg', 'Anil Kulkarni',
 'CS101', 'Programming in C', 'I03', 'Rahul Patil', 78),
('S01', 'Aarav Sharma', 'CSE', 'Computer Engg', 'Anil Kulkarni',
 'CS301', 'DBMS', 'I01', 'Anil Kulkarni', 91),
('S02', 'Isha Gupta', 'CSE', 'Computer Engg', 'Anil Kulkarni',
 'CS101', 'Programming in C', 'I03', 'Rahul Patil', 88),
('S02', 'Isha Gupta', 'CSE', 'Computer Engg', 'Anil Kulkarni',
 'CS301', 'DBMS', 'I01', 'Anil Kulkarni', 74),
('S03', 'Rohan Mehta', 'AIML', 'AI and ML', 'Meera Joshi',
 'CS101', 'Programming in C', 'I03', 'Rahul Patil', 65),
('S03', 'Rohan Mehta', 'AIML', 'AI and ML', 'Meera Joshi',
 'AI201', 'Python for DS', 'I04', 'Sneha Deshmukh', 81);

```

Q2: Verify the functional dependencies and measure the redundancy

```

SELECT 'roll_no -> student_name, dept_code' AS fd, COUNT(*) AS violations
FROM (SELECT roll_no FROM result_1nf GROUP BY roll_no
      HAVING COUNT(DISTINCT student_name, dept_code) > 1) t
UNION ALL
SELECT 'dept_code -> dept_name, hod_name', COUNT(*)
FROM (SELECT dept_code FROM result_1nf GROUP BY dept_code
      HAVING COUNT(DISTINCT dept_name, hod_name) > 1) t
UNION ALL
SELECT 'course_id -> course_title, instr_id', COUNT(*)
FROM (SELECT course_id FROM result_1nf GROUP BY course_id
      HAVING COUNT(DISTINCT course_title, instr_id) > 1) t
UNION ALL
SELECT 'instr_id -> instr_name', COUNT(*)
FROM (SELECT instr_id FROM result_1nf GROUP BY instr_id
      HAVING COUNT(DISTINCT instr_name) > 1) t;

SELECT dept_code, dept_name, hod_name, COUNT(*) AS times_stored
FROM result_1nf GROUP BY dept_code, dept_name, hod_name;

```

A query that finds a determinant value associated with more than one dependent value would prove that the dependency does not hold; zero violations confirm the FDs for this data.

Q3: Demonstrate update, insertion and deletion anomalies in the 1NF relation

```
START TRANSACTION;
UPDATE result_1nf SET dept_name = 'Computer Science' WHERE roll_no = 'S01';
SELECT DISTINCT dept_code, dept_name FROM result_1nf ORDER BY dept_code;
ROLLBACK;

INSERT INTO result_1nf (roll_no, course_id, course_title, instr_id, instr_name)
VALUES (NULL, 'CS401', 'Advanced Databases', 'I01', 'Anil Kulkarni');

START TRANSACTION;
DELETE FROM result_1nf WHERE roll_no = 'S03' AND course_id = 'AI201';
SELECT COUNT(*) AS rows_about_AI201_or_I04 FROM result_1nf
WHERE course_id = 'AI201' OR instr_id = 'I04';
ROLLBACK;
```

Each anomaly is demonstrated inside a transaction and undone with ROLLBACK so that the original data is preserved for the next steps.

Q4: Second Normal Form: remove partial dependencies on the key (roll_no, course_id)

```
CREATE TABLE student_2nf (
    roll_no CHAR(3) PRIMARY KEY, student_name VARCHAR(30),
    dept_code VARCHAR(5), dept_name VARCHAR(30), hod_name VARCHAR(30));
CREATE TABLE course_2nf (
    course_id CHAR(5) PRIMARY KEY, course_title VARCHAR(30),
    instr_id CHAR(3), instr_name VARCHAR(30));
CREATE TABLE result_2nf (
    roll_no CHAR(3), course_id CHAR(5), marks INT,
    PRIMARY KEY (roll_no, course_id),
    FOREIGN KEY (roll_no) REFERENCES student_2nf (roll_no),
    FOREIGN KEY (course_id) REFERENCES course_2nf (course_id));

INSERT INTO student_2nf
SELECT DISTINCT roll_no, student_name, dept_code, dept_name, hod_name
FROM result_1nf;
INSERT INTO course_2nf
SELECT DISTINCT course_id, course_title, instr_id, instr_name FROM result_1nf;
INSERT INTO result_2nf SELECT roll_no, course_id, marks FROM result_1nf;

SELECT * FROM student_2nf;
SELECT * FROM course_2nf;
```

Q5: Third Normal Form: remove transitive dependencies

```

CREATE TABLE department (
    dept_code VARCHAR(5) PRIMARY KEY, dept_name VARCHAR(30), hod_name VARCHAR(30));
CREATE TABLE instructor (
    instr_id CHAR(3) PRIMARY KEY, instr_name VARCHAR(30));
CREATE TABLE student (
    roll_no CHAR(3) PRIMARY KEY, student_name VARCHAR(30), dept_code VARCHAR(5),
    FOREIGN KEY (dept_code) REFERENCES department (dept_code));
CREATE TABLE course (
    course_id CHAR(5) PRIMARY KEY, course_title VARCHAR(30), instr_id CHAR(3),
    FOREIGN KEY (instr_id) REFERENCES instructor (instr_id));
CREATE TABLE result (
    roll_no CHAR(3), course_id CHAR(5), marks INT,
    PRIMARY KEY (roll_no, course_id),
    FOREIGN KEY (roll_no) REFERENCES student (roll_no),
    FOREIGN KEY (course_id) REFERENCES course (course_id));

INSERT INTO department SELECT DISTINCT dept_code, dept_name, hod_name FROM student_2nf;
INSERT INTO instructor SELECT DISTINCT instr_id, instr_name FROM course_2nf;
INSERT INTO student SELECT roll_no, student_name, dept_code FROM student_2nf;
INSERT INTO course SELECT course_id, course_title, instr_id FROM course_2nf;
INSERT INTO result SELECT * FROM result_2nf;

SELECT * FROM department;
SELECT * FROM instructor;
SELECT * FROM student;
SELECT * FROM course;

```

Q6: Verify that the decomposition is lossless (natural join gives back RESULT_1NF)

```

SELECT COUNT(*) AS rows_after_join
FROM result r
JOIN student s ON s.roll_no = r.roll_no
JOIN department d ON d.dept_code = s.dept_code
JOIN course c ON c.course_id = r.course_id
JOIN instructor i ON i.instr_id = c.instr_id;

SELECT * FROM result_1nf
EXCEPT
SELECT r.roll_no, s.student_name, d.dept_code, d.dept_name, d.hod_name,
c.course_id, c.course_title, i.instr_id, i.instr_name, r.marks
FROM result r
JOIN student s ON s.roll_no = r.roll_no
JOIN department d ON d.dept_code = s.dept_code
JOIN course c ON c.course_id = r.course_id
JOIN instructor i ON i.instr_id = c.instr_id;

```

EXCEPT (available in MySQL from 8.0.31) returns the rows of the original relation that are missing from the join. An empty result together with the equal row count (6) shows that no tuple is lost and no spurious tuple is generated.

Q7: Verify that the anomalies are eliminated in the 3NF schema

```

UPDATE department SET dept_name = 'Computer Science' WHERE dept_code = 'CSE';
SELECT s.roll_no, d.dept_name
FROM student s JOIN department d ON d.dept_code = s.dept_code;

INSERT INTO instructor VALUES ('I05', 'Pooja Kale');
INSERT INTO course VALUES ('CS401', 'Advanced Databases', 'I01');

DELETE FROM result WHERE roll_no = 'S03' AND course_id = 'AI201';
SELECT c.course_id, c.course_title, i.instr_name
FROM course c JOIN instructor i ON i.instr_id = c.instr_id
WHERE c.course_id IN ('AI201', 'CS401');

```


Q8: BCNF: relation TEACHING(student, subject, instructor) with instructor -> subject

```

CREATE TABLE teaching (
    student    CHAR(3),
    subject    VARCHAR(10),
    instructor VARCHAR(30),
    PRIMARY KEY (student, subject)
);
INSERT INTO teaching VALUES
    ('S01', 'DBMS', 'Anil Kulkarni'),
    ('S01', 'Python', 'Sneha Deshmukh'),
    ('S02', 'DBMS', 'Rahul Patil'),
    ('S03', 'DBMS', 'Anil Kulkarni'),
    ('S03', 'Python', 'Sneha Deshmukh');

START TRANSACTION;
INSERT INTO teaching VALUES ('S04', 'Java', 'Anil Kulkarni');
SELECT instructor, GROUP_CONCAT(DISTINCT subject) AS subjects
FROM   teaching GROUP BY instructor HAVING COUNT(DISTINCT subject) > 1;
ROLLBACK;

CREATE TABLE instructor_subject (
    instructor VARCHAR(30) PRIMARY KEY,
    subject    VARCHAR(10) NOT NULL);
CREATE TABLE student_instructor (
    student    CHAR(3),
    instructor VARCHAR(30),
    PRIMARY KEY (student, instructor),
    FOREIGN KEY (instructor) REFERENCES instructor_subject (instructor));
INSERT INTO instructor_subject SELECT DISTINCT instructor, subject FROM teaching;
INSERT INTO student_instructor SELECT student, instructor FROM teaching;

INSERT INTO instructor_subject VALUES ('Anil Kulkarni', 'Java');

SELECT * FROM teaching
EXCEPT
SELECT si.student, isub.subject, si.instructor
FROM   student_instructor si JOIN instructor_subject isub USING (instructor);
SELECT * FROM instructor_subject;

```

In TEACHING the inconsistent fact 'Anil Kulkarni teaches Java' is accepted because the determinant instructor is not a key. After BCNF decomposition the same fact is rejected by the primary key of INSTRUCTOR_SUBJECT, and the empty EXCEPT result shows that the decomposition is lossless.

Q9: 4NF: relation COURSE_BOOK(course_id, instructor, textbook) with multivalued dependencies

```

CREATE TABLE course_book (
    course_id CHAR(5),
    instructor VARCHAR(30),
    textbook   VARCHAR(20),
    PRIMARY KEY (course_id, instructor, textbook)
);
INSERT INTO course_book VALUES
    ('CS301', 'Anil Kulkarni', 'Silberschatz'),
    ('CS301', 'Anil Kulkarni', 'Elmasri-Navathe'),
    ('CS301', 'Rahul Patil', 'Silberschatz'),
    ('CS301', 'Rahul Patil', 'Elmasri-Navathe'),
    ('AI301', 'Meera Joshi', 'Tom Mitchell'),
    ('AI301', 'Meera Joshi', 'Aurelien Geron');

SELECT course_id, COUNT(*) AS rows_stored,
       COUNT(DISTINCT instructor) AS instructors, COUNT(DISTINCT textbook) AS textbooks

```

```

FROM   course_book GROUP BY course_id;

CREATE TABLE course_instructor (
    course_id CHAR(5), instructor VARCHAR(30), PRIMARY KEY (course_id, instructor));
CREATE TABLE course_textbook (
    course_id CHAR(5), textbook VARCHAR(20), PRIMARY KEY (course_id, textbook));
INSERT INTO course_instructor SELECT DISTINCT course_id, instructor FROM course_book;
INSERT INTO course_textbook   SELECT DISTINCT course_id, textbook   FROM course_book;

SELECT COUNT(*) AS rows_after_join
FROM   course_instructor ci JOIN course_textbook ct USING (course_id);
SELECT * FROM course_book
EXCEPT
SELECT ci.course_id, ci.instructor, ct.textbook
FROM   course_instructor ci JOIN course_textbook ct USING (course_id);

INSERT INTO course_textbook VALUES ('CS301', 'C. J. Date');
SELECT ci.course_id, ci.instructor, ct.textbook
FROM   course_instructor ci JOIN course_textbook ct USING (course_id)
WHERE  ci.course_id = 'CS301'
ORDER BY ci.instructor, ct.textbook;

```

In COURSE_BOOK a third CS301 textbook would need one new row for every CS301 instructor. After the 4NF decomposition it is a single row in COURSE_TEXTBOOK, and the join automatically pairs it with both instructors.

8. Sample Input / Database Tables

Unnormalized relation (UNF): result register with a repeating group

roll_no	student_name	dept_code	dept_name	hod_name	courses (course_id, title, instructor, marks)
S01	Aarav Sharma	CSE	Computer Engg	Anil Kulkarni	CS101, Programming in C, I03 Rahul Patil, 78; CS301, DBMS, I01 Anil Kulkarni, 91
S02	Isha Gupta	CSE	Computer Engg	Anil Kulkarni	CS101, Programming in C, I03 Rahul Patil, 88; CS301, DBMS, I01 Anil Kulkarni, 74
S03	Rohan Mehta	AIML	AI and ML	Meera Joshi	CS101, Programming in C, I03 Rahul Patil, 65; AI201, Python for DS, I04 Sneha Deshmukh, 81

The 1NF relation obtained by flattening the repeating group, as stored in MySQL by Q1 (read from the database after executing Q1):

Table: result_1nf

roll_n o	student_n ame	dept_co de	dept_name	hod_name	course_ id	course_title	instr_i d	instr_name	mar ks
S01	Aarav Sharma	CSE	Computer Engg	Anil Kulkarni	CS101	Programming in C	I03	Rahul Patil	78
S01	Aarav Sharma	CSE	Computer Engg	Anil Kulkarni	CS301	DBMS	I01	Anil Kulkarni	91
S02	Isha Gupta	CSE	Computer Engg	Anil Kulkarni	CS101	Programming in C	I03	Rahul Patil	88
S02	Isha Gupta	CSE	Computer Engg	Anil Kulkarni	CS301	DBMS	I01	Anil Kulkarni	74
S03	Rohan Mehta	AIML	AI and ML	Meera Joshi	AI201	Python for DS	I04	Sneha Deshmukh	81
S03	Rohan Mehta	AIML	AI and ML	Meera Joshi	CS101	Programming in C	I03	Rahul Patil	65

Table: teaching (BCNF example, Q8)

student	subject	instructor
S01	DBMS	Anil Kulkarni
S01	Python	Sneha Deshmukh
S02	DBMS	Rahul Patil
S03	DBMS	Anil Kulkarni
S03	Python	Sneha Deshmukh

Table: course_book (4NF example, Q9)

course_id	instructor	textbook
CS301	Anil Kulkarni	Silberschatz
CS301	Anil Kulkarni	Elmasri-Navathe
CS301	Rahul Patil	Silberschatz
CS301	Rahul Patil	Elmasri-Navathe
AI301	Meera Joshi	Tom Mitchell
AI301	Meera Joshi	Aurelien Geron

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Create the 1NF relation RESULT_1NF and load the sample data

```
mysql> DROP DATABASE IF EXISTS normalization_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE normalization_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE result_1nf (
->     roll_no      CHAR(3),
->     student_name VARCHAR(30),
->     dept_code    VARCHAR(5),
-> ...
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO result_1nf VALUES
-> ('S01', 'Aarav Sharma', 'CSE', 'Computer Engg', 'Anil Kulkarni',
->  'CS101', 'Programming in C', 'I03', 'Rahul Patil', 78),
-> ('S01', 'Aarav Sharma', 'CSE', 'Computer Engg', 'Anil Kulkarni',
-> ...
->  'AI201', 'Python for DS', 'I04', 'Sneha Deshmukh', 81);
Query OK, 6 rows affected (0.00 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

Output of Q2: Verify the functional dependencies and measure the redundancy

```
mysql> SELECT 'roll_no -> student_name, dept_code' AS fd, COUNT(*) AS violations
-> FROM (SELECT roll_no FROM result_1nf GROUP BY roll_no
->        HAVING COUNT(DISTINCT student_name, dept_code) > 1) t
-> UNION ALL
-> ...
->        HAVING COUNT(DISTINCT instr_name) > 1) t;
+-----+-----+
| fd | violations |
+-----+-----+
| roll_no -> student_name, dept_code | 0 |
| dept_code -> dept_name, hod_name | 0 |
| course_id -> course_title, instr_id | 0 |
| instr_id -> instr_name | 0 |
+-----+-----+
4 rows in set (0.01 sec)

mysql> SELECT dept_code, dept_name, hod_name, COUNT(*) AS times_stored
-> FROM result_1nf GROUP BY dept_code, dept_name, hod_name;
+-----+-----+-----+-----+
| dept_code | dept_name | hod_name | times_stored |
+-----+-----+-----+-----+
| CSE | Computer Engg | Anil Kulkarni | 4 |
| AIML | AI and ML | Meera Joshi | 2 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Output of Q3: Demonstrate update, insertion and deletion anomalies in the 1NF relation

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE result_1nf SET dept_name = 'Computer Science' WHERE roll_no = 'S01';
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> SELECT DISTINCT dept_code, dept_name FROM result_1nf ORDER BY dept_code;
+-----+-----+
| dept_code | dept_name |
+-----+-----+
| AIML | AI and ML |
| CSE | Computer Engg |
| CSE | Computer Science |
+-----+-----+
```

```

3 rows in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO result_1nf (roll_no, course_id, course_title, instr_id, instr_name)
-> VALUES (NULL, 'CS401', 'Advanced Databases', 'I01', 'Anil Kulkarni');
ERROR 1048 (23000) at line 55: Column 'roll_no' cannot be null

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM result_1nf WHERE roll_no = 'S03' AND course_id = 'AI201';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT COUNT(*) AS rows_about_AI201_or_I04 FROM result_1nf
-> WHERE course_id = 'AI201' OR instr_id = 'I04';
+-----+
| rows_about_AI201_or_I04 |
+-----+
| 0 |
+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

```

Output of Q4: Second Normal Form: remove partial dependencies on the key (roll_no, course_id)

```

mysql> CREATE TABLE student_2nf (
-> roll_no CHAR(3) PRIMARY KEY, student_name VARCHAR(30),
-> dept_code VARCHAR(5), dept_name VARCHAR(30), hod_name VARCHAR(30));
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course_2nf (
-> course_id CHAR(5) PRIMARY KEY, course_title VARCHAR(30),
-> instr_id CHAR(3), instr_name VARCHAR(30));
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE result_2nf (
-> roll_no CHAR(3), course_id CHAR(5), marks INT,
-> PRIMARY KEY (roll_no, course_id),
-> FOREIGN KEY (roll_no) REFERENCES student_2nf (roll_no),
-> FOREIGN KEY (course_id) REFERENCES course_2nf (course_id));
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO student_2nf
-> SELECT DISTINCT roll_no, student_name, dept_code, dept_name, hod_name
-> FROM result_1nf;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course_2nf
-> SELECT DISTINCT course_id, course_title, instr_id, instr_name FROM result_1nf;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO result_2nf SELECT roll_no, course_id, marks FROM result_1nf;
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM student_2nf;
+-----+-----+-----+-----+-----+
| roll_no | student_name | dept_code | dept_name | hod_name |
+-----+-----+-----+-----+-----+
| S01 | Aarav Sharma | CSE | Computer Engg | Anil Kulkarni |
| S02 | Isha Gupta | CSE | Computer Engg | Anil Kulkarni |
| S03 | Rohan Mehta | AIML | AI and ML | Meera Joshi |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```
mysql> SELECT * FROM course_2nf;
+-----+-----+-----+-----+
| course_id | course_title | instr_id | instr_name |
+-----+-----+-----+-----+
| AI201     | Python for DS | I04      | Sneha Deshmukh |
| CS101     | Programming in C | I03      | Rahul Patil |
| CS301     | DBMS          | I01      | Anil Kulkarni |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Output of Q5: Third Normal Form: remove transitive dependencies

```
mysql> CREATE TABLE department (
-> dept_code VARCHAR(5) PRIMARY KEY, dept_name VARCHAR(30), hod_name VARCHAR(30));
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE instructor (
-> instr_id CHAR(3) PRIMARY KEY, instr_name VARCHAR(30));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student (
-> roll_no CHAR(3) PRIMARY KEY, student_name VARCHAR(30), dept_code VARCHAR(5),
-> FOREIGN KEY (dept_code) REFERENCES department (dept_code));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE course (
-> course_id CHAR(5) PRIMARY KEY, course_title VARCHAR(30), instr_id CHAR(3),
-> FOREIGN KEY (instr_id) REFERENCES instructor (instr_id));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE result (
-> roll_no CHAR(3), course_id CHAR(5), marks INT,
-> PRIMARY KEY (roll_no, course_id),
-> FOREIGN KEY (roll_no) REFERENCES student (roll_no),
-> FOREIGN KEY (course_id) REFERENCES course (course_id));
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO department SELECT DISTINCT dept_code, dept_name, hod_name FROM student_2nf;
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor SELECT DISTINCT instr_id, instr_name FROM course_2nf;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student SELECT roll_no, student_name, dept_code FROM student_2nf;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course SELECT course_id, course_title, instr_id FROM course_2nf;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO result SELECT * FROM result_2nf;
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM department;
+-----+-----+-----+
| dept_code | dept_name | hod_name |
+-----+-----+-----+
| AIML      | AI and ML | Meera Joshi |
| CSE       | Computer Engg | Anil Kulkarni |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM instructor;
+-----+-----+
| instr_id | instr_name |
+-----+-----+
| I01      | Anil Kulkarni |
```

```

| I03      | Rahul Patil   |
| I04      | Sneha Deshmukh |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM student;
+-----+-----+-----+
| roll_no | student_name | dept_code |
+-----+-----+-----+
| S01     | Aarav Sharma | CSE       |
| S02     | Isha Gupta   | CSE       |
| S03     | Rohan Mehta  | AIML      |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM course;
+-----+-----+-----+
| course_id | course_title   | instr_id |
+-----+-----+-----+
| AI201     | Python for DS  | I04      |
| CS101     | Programming in C | I03      |
| CS301     | DBMS           | I01      |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

Output of Q6: Verify that the decomposition is lossless (natural join gives back RESULT_1NF)

```

mysql> SELECT COUNT(*) AS rows_after_join
-> FROM    result r
-> JOIN    student s    ON s.roll_no = r.roll_no
-> JOIN    department d ON d.dept_code = s.dept_code
-> JOIN    course c     ON c.course_id = r.course_id
-> JOIN    instructor i ON i.instr_id = c.instr_id;
+-----+
| rows_after_join |
+-----+
|                6 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM result_1nf
-> EXCEPT
-> SELECT r.roll_no, s.student_name, d.dept_code, d.dept_name, d.hod_name,
->        c.course_id, c.course_title, i.instr_id, i.instr_name, r.marks
-> FROM    result r
-> JOIN    student s    ON s.roll_no = r.roll_no
-> JOIN    department d ON d.dept_code = s.dept_code
-> JOIN    course c     ON c.course_id = r.course_id
-> JOIN    instructor i ON i.instr_id = c.instr_id;
Empty set (0.00 sec)

```

Output of Q7: Verify that the anomalies are eliminated in the 3NF schema

```

mysql> UPDATE department SET dept_name = 'Computer Science' WHERE dept_code = 'CSE';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT s.roll_no, d.dept_name
-> FROM    student s JOIN department d ON d.dept_code = s.dept_code;
+-----+-----+
| roll_no | dept_name       |
+-----+-----+
| S03     | AI and ML       |
| S01     | Computer Science |
| S02     | Computer Science |
+-----+-----+
3 rows in set (0.00 sec)

mysql> INSERT INTO instructor VALUES ('I05', 'Pooja Kale');

```

```
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO course VALUES ('CS401', 'Advanced Databases', 'I01');
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM result WHERE roll_no = 'S03' AND course_id = 'AI201';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT c.course_id, c.course_title, i.instr_name
-> FROM   course c JOIN instructor i ON i.instr_id = c.instr_id
-> WHERE  c.course_id IN ('AI201', 'CS401');
+-----+-----+-----+
| course_id | course_title      | instr_name |
+-----+-----+-----+
| AI201     | Python for DS     | Sneha Deshmukh |
| CS401     | Advanced Databases | Anil Kulkarni  |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Output of Q8: BCNF: relation TEACHING(student, subject, instructor) with instructor -> subject

```
mysql> CREATE TABLE teaching (
->   student   CHAR(3),
->   subject   VARCHAR(10),
->   instructor VARCHAR(30),
->   PRIMARY KEY (student, subject)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO teaching VALUES
->   ('S01', 'DBMS', 'Anil Kulkarni'),
->   ('S01', 'Python', 'Sneha Deshmukh'),
->   ('S02', 'DBMS', 'Rahul Patil'),
->   ('S03', 'DBMS', 'Anil Kulkarni'),
->   ('S03', 'Python', 'Sneha Deshmukh');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO teaching VALUES ('S04', 'Java', 'Anil Kulkarni');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT instructor, GROUP_CONCAT(DISTINCT subject) AS subjects
-> FROM   teaching GROUP BY instructor HAVING COUNT(DISTINCT subject) > 1;
+-----+-----+
| instructor | subjects |
+-----+-----+
| Anil Kulkarni | DBMS,Java |
+-----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE instructor_subject (
->   instructor VARCHAR(30) PRIMARY KEY,
->   subject   VARCHAR(10) NOT NULL);
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE student_instructor (
->   student   CHAR(3),
->   instructor VARCHAR(30),
->   PRIMARY KEY (student, instructor),
->   FOREIGN KEY (instructor) REFERENCES instructor_subject (instructor));
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO instructor_subject SELECT DISTINCT instructor, subject FROM teaching;
Query OK, 3 rows affected (0.00 sec)
```



```

Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_instructor SELECT student, instructor FROM teaching;
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor_subject VALUES ('Anil Kulkarni', 'Java');
ERROR 1062 (23000) at line 172: Duplicate entry 'Anil Kulkarni' for key
'instructor_subject.PRIMARY'

mysql> SELECT * FROM teaching
-> EXCEPT
-> SELECT si.student, isub.subject, si.instructor
-> FROM student_instructor si JOIN instructor_subject isub USING (instructor);
Empty set (0.00 sec)

mysql> SELECT * FROM instructor_subject;
+-----+-----+
| instructor | subject |
+-----+-----+
| Anil Kulkarni | DBMS |
| Rahul Patil | DBMS |
| Sneha Deshmukh | Python |
+-----+-----+
3 rows in set (0.00 sec)

```

Output of Q9: 4NF: relation COURSE_BOOK(course_id, instructor, textbook) with multivalued dependencies

```

mysql> CREATE TABLE course_book (
-> course_id CHAR(5),
-> instructor VARCHAR(30),
-> textbook VARCHAR(20),
-> PRIMARY KEY (course_id, instructor, textbook)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO course_book VALUES
-> ('CS301', 'Anil Kulkarni', 'Silberschatz'),
-> ('CS301', 'Anil Kulkarni', 'Elmasri-Navathe'),
-> ('CS301', 'Rahul Patil', 'Silberschatz'),
-> ('CS301', 'Rahul Patil', 'Elmasri-Navathe'),
-> ('AI301', 'Meera Joshi', 'Tom Mitchell'),
-> ('AI301', 'Meera Joshi', 'Aurelien Geron');
Query OK, 6 rows affected (0.00 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> SELECT course_id, COUNT(*) AS rows_stored,
-> COUNT(DISTINCT instructor) AS instructors, COUNT(DISTINCT textbook) AS textbooks
-> FROM course_book GROUP BY course_id;
+-----+-----+-----+-----+
| course_id | rows_stored | instructors | textbooks |
+-----+-----+-----+-----+
| AI301 | 2 | 1 | 2 |
| CS301 | 4 | 2 | 2 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> CREATE TABLE course_instructor (
-> course_id CHAR(5), instructor VARCHAR(30), PRIMARY KEY (course_id, instructor));
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course_textbook (
-> course_id CHAR(5), textbook VARCHAR(20), PRIMARY KEY (course_id, textbook));
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO course_instructor SELECT DISTINCT course_id, instructor FROM course_book;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course_textbook SELECT DISTINCT course_id, textbook FROM course_book;

```

```

Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT COUNT(*) AS rows_after_join
-> FROM   course_instructor ci JOIN course_textbook ct USING (course_id);
+-----+
| rows_after_join |
+-----+
|                6 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM course_book
-> EXCEPT
-> SELECT ci.course_id, ci.instructor, ct.textbook
-> FROM   course_instructor ci JOIN course_textbook ct USING (course_id);
Empty set (0.00 sec)

mysql> INSERT INTO course_textbook VALUES ('CS301', 'C. J. Date');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT ci.course_id, ci.instructor, ct.textbook
-> FROM   course_instructor ci JOIN course_textbook ct USING (course_id)
-> WHERE  ci.course_id = 'CS301'
-> ORDER BY ci.instructor, ct.textbook;
+-----+-----+-----+
| course_id | instructor | textbook |
+-----+-----+-----+
| CS301     | Anil Kulkarni | C. J. Date |
| CS301     | Anil Kulkarni | Elmasri-Navathe |
| CS301     | Anil Kulkarni | Silberschatz |
| CS301     | Rahul Patil | C. J. Date |
| CS301     | Rahul Patil | Elmasri-Navathe |
| CS301     | Rahul Patil | Silberschatz |
+-----+-----+-----+
6 rows in set (0.00 sec)

```

10. Result

The result register was normalized from UNF to 1NF, 2NF and 3NF (five relations: DEPARTMENT, INSTRUCTOR, STUDENT, COURSE, RESULT), which are also in BCNF and 4NF. In MySQL the four FDs held with zero violations; the update anomaly (two names for CSE), insertion anomaly (ERROR 1048) and deletion anomaly (loss of AI201 and its instructor) occurred in the 1NF relation but not in the 3NF schema. The join of the 3NF relations returned exactly the 6 original tuples (EXCEPT gave an empty set). The TEACHING relation was decomposed into BCNF, after which the inconsistent fact was rejected by a primary key, and COURSE_BOOK was decomposed into 4NF, removing the redundancy caused by the multivalued dependencies.

11. Conclusion

Normalization removes redundancy by making every determinant a key: 2NF removes partial dependencies, 3NF removes transitive dependencies, BCNF removes every FD whose determinant is not a superkey, and 4NF removes independent multivalued facts stored together. The SQL experiments showed that anomalies present in the unnormalized design disappear after decomposition and that the decompositions are lossless. BCNF may sacrifice dependency preservation, so in practice 3NF/BCNF is chosen according to the dependencies that must be enforced, while 4NF is applied wherever independent multivalued attributes occur.